

National University of Computer and Emerging Sciences



Lab Manual 01

Computer Organization
and Assembly Language

Course Instructor	Mr. Salman Mubarik
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Fall 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Objectives:.....	4
Activities:	4
Activity 1 (Introduction to the Assembly Language Environment):.....	4
• Introduction	4
• DOSBox	4
Why do we need DOSBox?.....	4
• NASM	5
• AFD	5
• Installing and Setting Up the Required Tools	5
Step 1: Install DOSBox	5
• Creating the Tools Folder	6
• Starting DOSBox.....	6
○ Mounting the Tools Folder	6
• Changing to the Mounted Drive	6
• Assembling an Assembly Program.....	6
• Listing Files	6
○ What Is a Listing File?	7
• Mnemonic vs Opcode vs Machine Code	7
○ Mnemonic	7
○ Opcode	7
○ Machine Code	7
• Purpose of [ORG 0x0100]	7
Activity 2 (Writing and Running Your First Assembly Program):.....	8
• Writing the Program	8
• Comments in Assembly Language	8
• Saving the Program.....	8
• Assembling the Program	8
• Inspecting the Listing File.....	8
• Running the Program in AFD.....	9
○ Executing the Program Step by Step	9
○ Questions — AFD Debugging.....	9
○ Understanding IP	9
Activity 3 (Basic Assembly Programming Tasks):.....	9
1. Sum of the First Four Even Numbers.....	9

Approach 1 — Using Two Registers	9
Approach 2 — Using a Single Register	10
Submission	10
2. Rotate the Values of Three Registers.....	10
Submission:	10

Objectives:

- To introduce, setup and use of DOSBOX, NASM and AFD
- Running your first program in Assembly
- Practicing few basic programs in Assembly
- Explaining basic functions in debugger

Activities:

Activity 1 (Introduction to the Assembly Language Environment):

- *Introduction*

Assembly Language is a low-level programming language that provides a relatively human-readable representation of the instructions executed by a processor. Unlike high-level languages such as C++, Java, or Python, Assembly Language works very closely with the processor's:

- Registers
- Memory
- Arithmetic and logic operations
- Instruction Pointer
- Flags
- Stack

In this lab, we will work with an 8086-style Assembly Language environment using:

- DOSBox
- NASM
- AFD

- *DOSBox*

DOSBox is an emulator that recreates an MS-DOS-compatible environment. It provides an environment in which old DOS-based programs and applications can run on modern computers without requiring the original DOS operating system. DOSBox can emulate various aspects of the old DOS environment, including:

- Sound
- Input
- Graphics
- Basic networking
- DOS file-system behavior

It is widely used to run classic MS-DOS games and applications on modern operating systems.

Why do we need DOSBox?

Many older Assembly Language development and debugging tools were designed to run in an MS-DOS environment. Modern versions of Windows do not provide the same DOS environment expected by these old programs. DOSBox solves this problem by creating a virtual DOS environment. The important idea is:

DOSBox creates a virtual DOS environment rather than allowing an old DOS program to directly operate in the modern Windows environment.

- *NASM*

NASM stands for Netwide Assembler and is an assembler which converts Assembly Language source code into machine code that a processor can execute. For example, a programmer may write:

```
MOV AX, 5
```

```
ADD AX, 3
```

These instructions are written using human-readable Assembly Language mnemonics. NASM converts them into the corresponding machine-code bytes. NASM is commonly used for programming and learning x86/x86-64 Assembly Language. In this lab, we will use NASM to assemble our Assembly Language programs into .COM executable files.

- *AFD*

AFD stands for Assembly File Debugger and is a debugger for Assembly Language programs which allows us to execute a program step by step and observe what happens inside the processor. For example, we can inspect registers such as:

- AX
- BX
- CX
- DX
- IP
- FLAGS

We can also observe memory and stack information. Consider the following instructions:

```
MOV AX, 5
```

```
ADD AX, 3
```

Before executing ADD AX, 3:

```
AX = 0005
```

After executing ADD AX, 3:

```
AX = 0008
```

Hence, a debugger allows us to observe this change directly.

- *Installing and Setting Up the Required Tools*

The following files are provided through Google Classroom:

- DOSBox
- NASM
- AFD

You should download all required files before starting the lab.

Step 1: Install DOSBox

For an installer:

- Double-click the DOSBox installer.
- Follow the installation wizard.
- Click Next as required.
- Click Install.
- Complete the installation.

Alternatively, a portable version of DOSBox is provided.

If you use the portable version:

- Extract the .rar file.
- Locate the DOSBox application file.
- You can run DOSBox directly without performing a traditional installation.

- *Creating the Tools Folder*

Create a folder named 'Tools'. Choose a location for folder that is easy for you to remember. The Tools folder should contain the following files:

- nasm.exe
- afd.exe
- DOSBox application shortcut

Your Assembly source files should also be saved inside this folder while performing the lab.

- *Starting DOSBox*

Double-click the DOSBox application. DOSBox will open a command window similar to an old DOS environment. However, DOSBox initially does not automatically have access to your Windows Tools folder. Therefore, we must mount the folder.

- *Mounting the Tools Folder*

Suppose your Tools folder is located at:

D:\COAL\Tools

In DOSBox, type:

MOUNT C D:\COAL\Tools

After this command, DOSBox makes:

D:\COAL\Tools

appear inside the DOS environment as:

C:

The general syntax is:

MOUNT <virtual-drive> <Windows-folder-path>

The point to remember here is that the C in **MOUNT C D:\COAL\Tools** does not represent your physical Windows C: drive as it is the virtual drive letter that DOSBox will use for the mounted folder. Moreover, you could use another virtual drive letter as well.

For example: **MOUNT D D:\COAL\Tools** would make the Windows folder available as virtual drive D: inside DOSBox.

- *Changing to the Mounted Drive*

After mounting the folder, type **C:** and press Enter. You are now working inside the virtual C: drive. You can use the command **DIR** to view its contents. The DIR command lists files and folders available in the current directory.

- *Assembling an Assembly Program*

Suppose you have an Assembly Language source file named **hello.asm**. The basic NASM command is:

nasm hello.asm -o hello.com

This command tells NASM to Read hello.asm, Assemble it and Produce hello.com

The general form is:

nasm source.asm -o output.com

- *Listing Files*

NASM can also generate a listing file. A listing file is a text file that shows the Assembly source code together with information about the machine-code bytes generated for each instruction. To generate a listing file from **hello.asm**, use:

nasm hello.asm -l hello.lst

You can also generate the .COM and .LST files together:

nasm hello.asm -o hello.com -l hello.lst

- What Is a Listing File?

A listing file allows us to see the relationship between:

Assembly instruction

↓

Machine-code bytes

Suppose a program contains:

MOV AX, 5

ADD AX, 3

The listing file may contain information conceptually similar to:

00000000 B80500 MOV AX, 5

00000003 050300 ADD AX, 3

The exact output depends on the program and NASM configuration. The information can be understood conceptually as:

00000000 B80500 MOV AX, 5
^^^^^^^^ ^^^^^^ ^^^^^^^^^

address	machine code	Assembly instruction
00000000	B80500	MOV AX, 5
00000003	050300	ADD AX, 3

- *Mnemonic vs Opcode vs Machine Code*

- Mnemonic

A mnemonic is a human-readable name representing an Assembly operation. Examples include:

MOV
ADD
SUB
INC
DEC
CMP

Mnemonics are designed to make Assembly Language easier for humans to understand and remember. The CPU does not directly execute the words like MOV. The assembler converts the mnemonic and its operands into machine-code bytes.

- Opcode

An opcode is the part of the machine-code encoding that identifies the operation to be performed. For example, for the instruction:

MOV AX, 5

the opcode byte is:

B8

The complete machine-code encoding contains additional bytes representing the immediate value.

- Machine Code

Machine code is the binary/byte representation of instructions that the processor can execute. For example:

MOV AX, 5

may be represented by:

B8 05 00

- *Purpose of [ORG 0x0100]*

You will frequently see the following directive at the beginning of our programs:

[ORG 0x0100]

0x0100 is a hexadecimal number and its decimal value is 256. The ORG directive tells NASM what address/offset should be assumed as the starting location of the program when generating address information. For DOS .COM programs, execution begins at offset 0100h. The first 256 bytes of the DOS program segment are reserved for the Program Segment Prefix (PSP), which is created by DOS before the .COM program starts.

Activity 2 (Writing and Running Your First Assembly Program):

- *Writing the Program*

Open Notepad or another plain-text editor and write the following program:

```
; this is a comment. Comment starts with semicolon
; this program adds three numbers in registers
[org 0x0100]    ; we will see ORG directive later
mov ax, 5      ; AX = 5
mov bx, 10     ; BX = 10
add ax, bx     ; AX = AX + BX
mov bx, 15     ; BX = 15
add ax, bx     ; AX = AX + BX
mov ax, 0x4c00 ; terminate the program
int 0x21
```

- *Comments in Assembly Language*

In NASM syntax, a semicolon begins a comment. For example:

```
; This is a comment
```

You can also place comments after an instruction:

```
mov ax, 5    ; Store 5 in AX
```

Comments are ignored by the assembler and are intended for human readers. They help explain what the program is doing.

- *Saving the Program*

Save the program inside your Tools folder. For example:

```
D:\COAL\Tools\file_name.asm
```

Make sure that the file is saved as **file_name.asm** and not **file_name.asm.txt**. If you are using Notepad, select Save as type **All Files** when saving an Assembly source file.

- *Assembling the Program*

Start DOSBox and mount the Tools folder. For example: **MOUNT C D:\COAL\Tools**

Then: **C:**

Moreover, use **DIR** to verify that your source file is present.

Now assemble the program using:

```
nasm file_name.asm -o file_name.com -l file_name.lst
```

After successful assembly, you should have:

```
file_name.asm
file_name.com
file_name.lst
```

- *Inspecting the Listing File*

Open **file_name.lst** using Notepad. Carefully examine the generated machine-code bytes and offsets. Use the listing file to answer the following questions:

- What is the opcode of the instruction **MOV AX, someConstant**

- Verify the above opcode everywhere this instruction has been used in the program.
- What are all the machine-code bytes generated for the instructions in this program?
- Identify and verify the opcode of **MOV BX, someConstant** throughout the machine code.
- What is the offset of the first instruction?
- Why are the offsets of the second and third instructions separated by 3 bytes?

- **Running the Program in AFD**

After assembling the program, run **afd file_name.com**. This command opens AFD and loads **file_name.com** into the debugger. The debugger window will provide information about various components of the processor/program, such as:

Registers
Stack
Flags
Instruction Pointer
Program instructions

The exact arrangement of the debugger window may depend on the version/configuration of AFD provided for the course.

- **Executing the Program Step by Step**

Press F1 to execute the program one instruction at a time. Continue pressing F1 and observe how the register values change. Remember that the debugger display values in hexadecimal.

- **Questions — AFD Debugging**

While executing the program in AFD, answer the following:

- What is the value of the IP (Instruction Pointer) register when the program is initially loaded? What does this value indicate?
- Record the initial values of the data registers AX, BX, CX & DX.
- Press F1 continuously, one instruction at a time, and observe as well as record the values of data registers.

- **Understanding IP**

The Instruction Pointer (IP) contains the offset/address of the next instruction to be executed. For a .COM program, the first instruction begins at offset **0100h** because the first 256 bytes are occupied by the DOS Program Segment Prefix. Therefore, when you see the initial IP in AFD, relate it to **[ORG 0x0100]**. As instructions execute, IP advances to point to subsequent instructions. This is one of the most important concepts to observe in the debugger.

Activity 3 (Basic Assembly Programming Tasks):

1. *Sum of the First Four Even Numbers*

Write an Assembly Language program to calculate the sum of the first four even numbers: 2, 4, 6, 8. The expected result is 20. Your program must calculate the result using registers.

Approach 1 — Using Two Registers

You should implement the calculation using these two registers:

- One register for the running sum
- One register for the current even number

Approach 2 — Using a Single Register

Implement the same calculation using a single register, based on the assumption that an instruction such as **ADD AX, 2** is available in the instruction set/environment being used.

Submission:

For each approach, submit .asm, .lst and .com files. Moreover, You should also verify the execution of both programs in AFD.

2. *Rotate the Values of Three Registers*

Write an Assembly Language program that rotates the values of three registers counter-clockwise twice. Considering Initial values as:

AX = 5

BX = 15

CX = 25

You may use DX as an additional temporary register. First draw data registers on paper. Solve this problem on paper then write your code accordingly. After writing your code verify the execution on paper first then check the execution on AFD.

Submission:

Submit .asm, .lst and .com files for this problem as well.